Name: Benedicto, Ruel Jr A.	**Date Performed:** Feb. 26, 2026
Course/Section: CPE31S1	**Date Submitted:** March 10, 2026
Instructor: Engr. Robin Valenzuela	**Semester and SY:** 2nd semester 2025-2026

Activity 7: Managing Files and Creating Roles in Ansible

1. **Objectives:**

1.1 Manage files in remote servers

1.2 Implement roles in ansible

2. **Discussion**:

In this activity, we look at the concept of copying a file to a server. We are going to create a file into our git repository and use Ansible to grab that file and put it into a particular place so that we could do things like customize a default website, or maybe install a default configuration file. We will also implement roles to consolidate plays.

Task 1: Create a file and copy it to remote servers

1. Using the previous directory we created, created a directory, and named it "*files*." Create a file inside that directory and name it "*default_site.html*." Edit the file and put basic HTML syntax. Any content will do, as long as it will display text later. Save the file and exit.

2. Edit the *site.yml* file and just below the *web_servers* play, create a new file to copy the default html file for site:

```
- name: copy default html file for site

  tags: apache, apache2, httpd
  copy:
    src: default_site.html
    dest: /var/www/html/index.html
    owner: root
    group: root
    mode: 0644
```

3. Run the playbook *site.yml*. Describe the changes.

```
20
21  - hosts: web_servers
22    become: true
23    tasks:  #Task 1.1: Targetting Specific Nodes
24
25      - name: copy default html file for site.yml
26        tags: apache, apache, httpd
27        copy:
28          src: default_site.html
29          dest: /var/www/html/index.html
30          owner: root
31          group: root
32          mode: 0644
33
34      - name: Install apache and php for Ubuntu servers
35        tags: apache,apache2,ubuntu
36        apt:
```

PROBLEMS OUTPUT TERMINAL PORTS

```
benedicto@Workstation:~/CPE232_Benedicto$ ansible-playbook site.yml -K

PLAY [file_servers] ************************************************

TASK [Gathering Facts] *********************************************
ok: [Server3]

TASK [Install samba package] ***************************************
ok: [Server3]

PLAY RECAP *********************************************************
Server1                    : ok=5    changed=1    unreachable=0    faile
Server2                    : ok=4    changed=0    unreachable=0    faile
Server3                    : ok=4    changed=0    unreachable=0    faile

benedicto@Workstation:~/CPE232_Benedicto$
```

4. Go to the remote servers (*web_servers*) listed in your inventory. Use cat command to check if the index.html is the same as the local repository file (*default_site.html*). Do both for Ubuntu and CentOS servers. On the CentOS server, go to the browser and type its IP address. Describe the output.
5. Sync your local repository with GitHub and describe the changes.

```
benedicto@Server1:~$ cat /var/www/html/index.html
---
<!DOCTYPE html>
<html>
<head>
        <title>Ansible Test Page</title>
</head>

<body>
        <h1>hello from ansible</h1>
        <p>This page was deployed using an Ansible role.</p>
</body>
</html>
benedicto@Server1:~$
```

This is the output from the (web_servers), I don't have CentOS in my personal device, that's why it's only in Ubuntu but i'll pull the code from github to do it again in computer school.

Task 2: Download a file and extract it to a remote server

1. Edit the site.yml. Just before the web_servers play, create a new play:

```
- hosts: workstations
  become: true
  tasks:

  - name: install unzip
    package:
      name: unzip

  - name: install terraform
    unarchive:
      src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
      dest: /usr/local/bin
      remote_src: yes
      mode: 0755
      owner: root
      group: root
```

2. Edit the inventory file and add workstations group. Add any Ubuntu remote server. Make sure to remember the IP address.
3. Run the playbook. Describe the output.

- **I added [workstations] in the inventory.ini and copy-paste the server2 since I don't have CentOS yet and the output shows that the Server2 changed=1 meaning that the terraform is installed.**

```
21  - hosts: workstations #HOA7 Task 2: Download a file and extract it to a remote server
22    become: true
23    tasks:
24      - name: uninstall unzip
25        package:
26          name: unzip
27
28      - name: install terraform
29        unarchive:
30          src: https://releases.hashicorp.com/terraform/0.12.28/terraform_0.12.28_linux_amd64.zip
31          dest: /usr/local/bin
32          remote_src: yes
33          mode: 0755
34          owner: root
35          group: root
```

PROBLEMS OUTPUT TERMINAL PORTS

```
benedicto@Workstation:~/CPE232_Benedicto$ ansible-playbook site.yml -K
TASK [Gathering Facts] *********************************************************************************
ok: [Server3]

TASK [Install samba package] ***************************************************************************
ok: [Server3]

PLAY RECAP *********************************************************************************************
Server1                    : ok=5    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignc
Server2                    : ok=7    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignc
Server3                    : ok=4    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignc

benedicto@Workstation:~/CPE232_Benedicto$
```

Ln 21, Col 22 Spaces: 2

4. On the Ubuntu remote workstation, type terraform to verify installation of terraform. Describe the output.

- **The output shows the version of the terraform and it recommends updating it to the latest version.**

```
benedicto@Server2:~$ terraform version
Terraform v0.12.28

Your version of Terraform is out of date! The latest version
is 1.14.6. You can update by downloading from https://www.terraform.io/downloads
.html
benedicto@Server2:~$
```

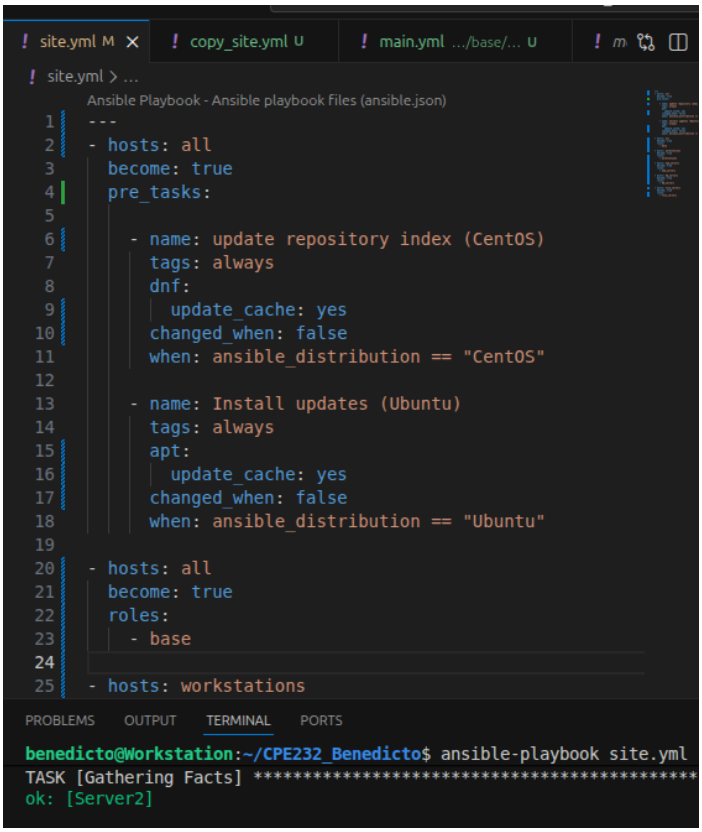
Task 3: Create roles

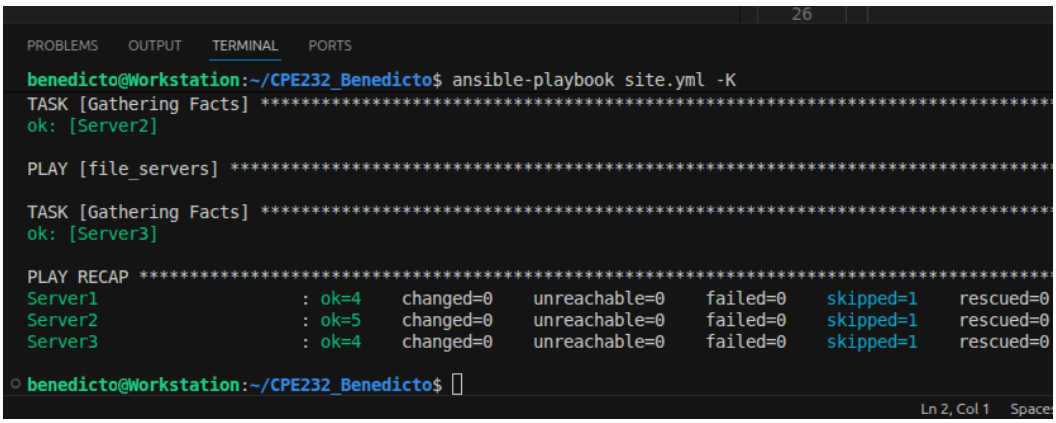
1. Edit the site.yml. Configure roles as follows: (make sure to create a copy of the old site.yml file because you will be copying the specific plays for all groups)

```
---
- hosts: all
  become: true
  pre_tasks:

  - name: update repository index (CentOS)
    tags: always
    dnf:
      update_cache: yes
    changed_when: false
    when: ansible_distribution == "CentOS"
  - name: install updates (Ubuntu)
    tags: always
    apt:
      update_cache: yes
    changed_when: false
    when: ansible_distribution == "Ubuntu"

- hosts: all
  become: true
  roles:
    -  base

- hosts: workstations
  become: true
  roles:
    - workstations

- hosts: web_servers
  become: true
  roles:
    - web_servers

- hosts: db_servers
  become: true
  roles:
    - db_servers

- hosts: file_servers
  become: true
  roles:
    - file_servers
```

Save the file and exit.

2. Under the same directory, create a new directory and name it roles. Enter the roles directory and create new directories: base, web_servers, file_servers, db_servers and workstations. For each directory, create a directory and name it tasks.
3. Go to tasks for all directory and create a file. Name it main.yml. In each of the tasks for all directories, copy and paste the code from the old site.yml file. Show all contents of main.yml files for all tasks.
4. Run the site.yml playbook and describe the output.





Reflections:

Answer the following:

1. What is the importance of creating roles?
   - **Creating roles is important because it organizes Ansible tasks into reusable and structured components. It makes automation easier to manage, reuse, and maintain, especially in large projects.**
2. What is the importance of managing files?
   - **Managing files is important because it ensures configurations and system files are consistent and correctly deployed across machines. It helps maintain system reliability and reduces manual errors.**